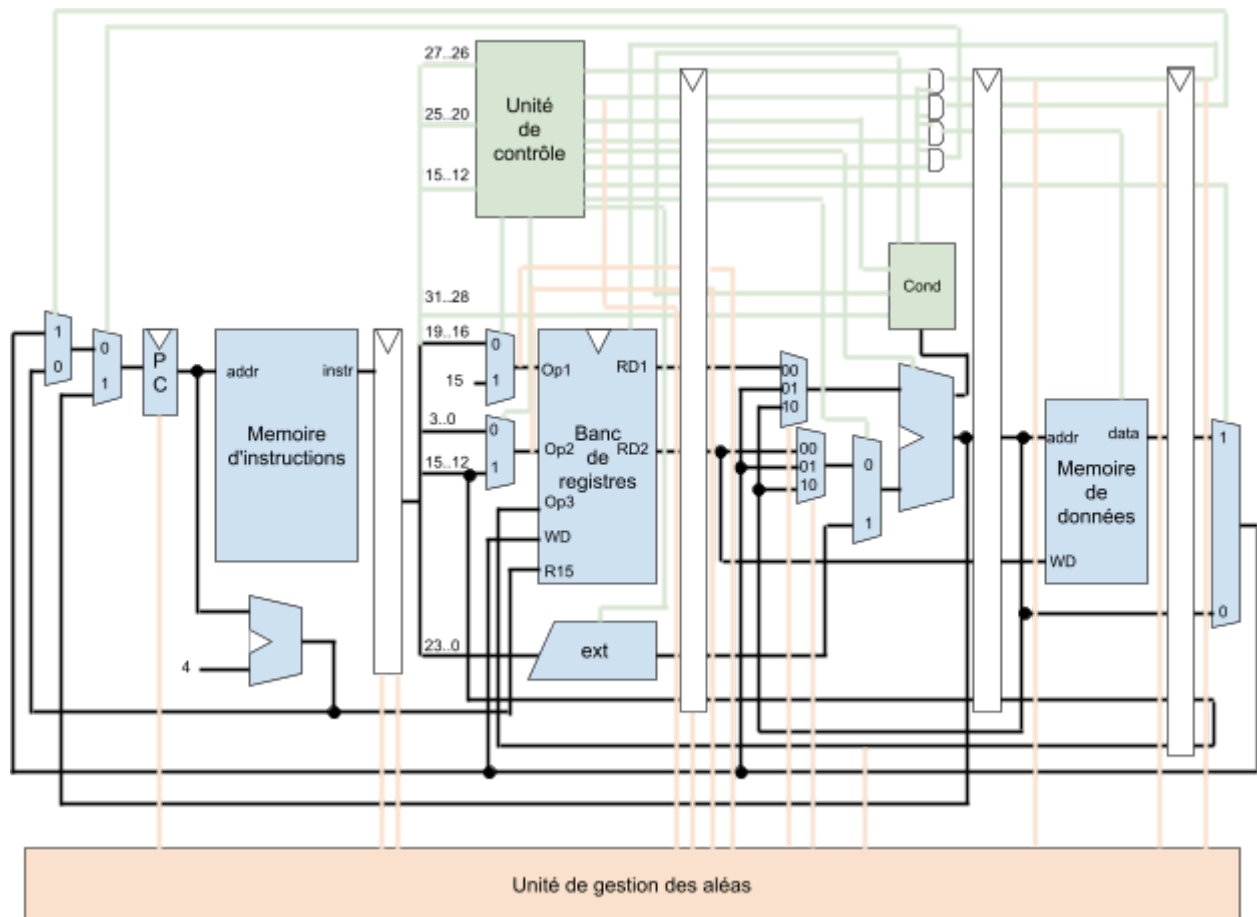


TP MACM (partie 3) – M1 SECIL

L'objectif de ces 5 séances de TP est de décrire en VHDL un processeur pipeliné dont le jeu d'instructions est basé sur l'ISA ARM (très simplifiée). On commencera par réaliser le chemin de données à partir de composants VHDL fournis, puis on développera la logique de contrôle. Enfin on rajoutera la logique de gestion des aléas de données.



III. Troisième partie : unité de gestion des aléas

Nous finissons l'implantation de notre processeur, en développant l'unité de gestion des aléas.

III.1. Forwarding des opérandes (aléas de données)

Nous commençons par la gestion du forwarding des opérandes. Il nous faut détecter lorsque l'une des opérandes (ou les deux) de l'opération dans l'étage d'exécution est calculée par une instruction couramment dans l'étage mémoire ou retire. Pour ce faire, on compare la valeur des signaux `a1` et `a2` (sorties de registres entre l'étage DE et EX) avec la valeur des signaux `Op3_ME_out` (produit à l'étage ME) et `Op3_RE_out` (produit à l'étage RE), tout en s'assurant que les instructions correspondantes réalisent bien une écriture dans les registres. On donne la priorité à la comparaison avec `Op3_ME_out`, car en cas d'égalité entre `Op3_ME_out` et `Op3_RE_out`, c'est `Op3_ME_out` qui représente la valeur la plus "fraîche". Le résultat de la comparaison permet de générer les signaux de contrôle `EA_EX` et `EB_EX` des deux multiplexeurs à 3 entrées de l'étage EX. En cas d'égalité, on vérifiera que l'instruction correspondante réalise bien une écriture dans les registres, en regardant la valeur du signal `RegWr` correspondant à l'instruction qui produit la valeur de l'opérande (c'est à dire la valeur de `RegWr` à l'étage ME ou RE selon). On nommera ces deux signaux `RegWr_Mem` et `RegWr_RE`.

`EA_EX` vaut 10 si `a1 = Op3_ME_out` et que `RegWr_Mem = 1`. `EA_EX` vaut 01 si `a1 != Op3_ME_out` et `a1 = Op3_RE_out` et `RegWr_RE = 1`. Enfin dans tous les autres cas, `EA_EX` vaut 00.

Les mêmes formules se transposent à `EB_EX` en prenant `a2` comme entrée.

III.2. Retardement des instructions en cas de forwarding depuis la mémoire

Lorsque la valeur à forwarder provient d'une instruction mémoire, cette valeur n'est pas prête à être forwardée au cycle courant, depuis l'étage ME. Il faut attendre que la mémoire ait répondu, et que la valeur soit passée dans l'étage RE. Pour réaliser cette attente, on va chercher à détecter l'aléa dès l'étage de décodage et à insérer un NOP juste après l'instruction mémoire.

On va pour cela générer un signal `LDRStall` à partir des indices des registres opérandes dans l'étage de décodage et du signal de contrôle `MemToReg` dans l'étage d'exécution (qui témoigne que l'instruction en cours d'exécution est un LDR):

`LDRStall = 1` si (`Reg1 = Op3_EX_out` ou `Reg2 = Op3_EX_out`) et `MemToReg_EX = 1`.

A partir du signal `LDRStall`, on peut générer les signaux `Gel_Li`, `En_DI` et `Clr_EX` qui valent tous l'inverse de `LDRStall`. `Gel_LI` et `EN_DI` permettent de geler les registres des étages FE et DE pendant un cycle (et donc de maintenir l'instruction courante dans l'étage DE et la suivante dans l'étage FE), et `Clr_EX` génère un NOP pour le cycle suivant.

III.3. Aléas de contrôle

Nous devons désormais gérer les aléas de contrôle: lorsqu'une instruction modifiant le pc sont exécutés, les instructions qui les suivent dans le pipeline ne doivent pas être exécutées. Il nous faut implémenter un moyen de les "flush".

En cas de branchement, l'adresse de la cible du branchement est connue dans l'étage d'exécution et est forwardée vers l'étage de fetch. Il nous faut donc transformer en nop les instructions couramment dans l'étage FE et DE. Pour cela on doit activer les entrées clear (synchrones) des buffers situés entre les étages FE et DE et entre les étages DE et EX.

On aura donc : $\text{Clr_EX} = \text{Non (LDRStall + Bpris_EX)}$

En cas d'instruction modifiant le PC, le signal PCSrc et ses déclinaisons dans les différents étages passent à 1. On détecte une telle instruction dès l'étage de décodage. Il faut alors stall l'étage FE (pour conserver en place la prochaine instruction au cas où la condition d'exécution de l'instruction modifiant le PC ne soit pas valide), et insérer des nop à partir de l'étage de décodage tant que l'instruction modifiant le PC n'est pas sortie du pipeline (ou n'a pas été invalidée par sa condition). On va donc bloquer le PC dans l'étage FE, et activer les entrées clr du registre entre les étages FE et DE dès lors que l'on découvre une instruction qui va modifier le PC. Toutefois, on va dégeler le PC dans l'étage FE lorsque l'instruction modifiant le PC arrive dans l'étage RE, afin que la mise à jour se fasse avec la valeur provenant de RE.

On aura donc : $\text{GEL_LI} = \text{NON (LDRStall + PCSrc_DE + PCSrc_EX + PCSrc_ME)}$

Et : $\text{Clr_DI} = \text{NON (PCSrc_DE + PCSrc_EX + PCSrc_ME + PCSrc_ER + Bpris_EX)}$